ToddySoft Connect ctrlX Driver - Ignition 8.3

Ignition 8.3 module for connecting to Bosch Rexroth **ctrlX CORE** controllers over the ctrlX Data Layer's HTTPS REST API. Every node the Data Layer exposes is reachable the same way — PLC symbols, axis state, framework metrics — addressed by its slash-separated node path.

The device configuration uses a custom configuration form: settings that do not apply to what you have selected stay out of your way, and the fields that carry a security consequence say so where you read them.

> **This is an HTTPS management interface, not a fieldbus.** A tag group's tags are read together, one HTTPS request per group and poll, but each is a round trip to the controller's web server, so this driver suits diagnostics, configuration and moderate-rate supervision rather than high-rate process data. Size your tag groups accordingly; see *Tag Groups*.

Installation

> **This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `…-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. This module declares the dependency as **required** — a mechanism 8.3 has and 8.1 lacks — so Ignition does not attempt to load it at all, and reports the reason instead of failing with a stack trace. In **Status > Diagnostics > Logs** you will see:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> This is the same handling Ignition's own drivers get in the same situation.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**.
3. Select the `ToddySoft-Connect-CtrlX-Driver-8.3.modl` file and upload it.
4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.

Before you start: the controller side

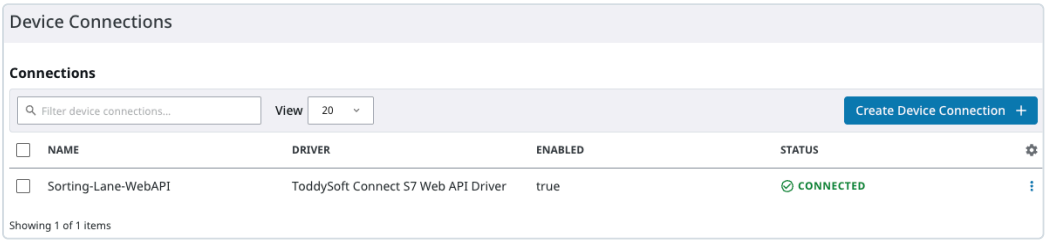
Two things must be true on the ctrlX CORE before a connection can succeed.

- **An account with the right role.** ctrlX manages read and write as separate permissions, so an account that browses and reads happily may still refuse every write with HTTP `403`. Roles are assigned in the controller's own user management.
- **The Data Layer service running**, and HTTPS reachable on port 443 (or whatever you override the port to).

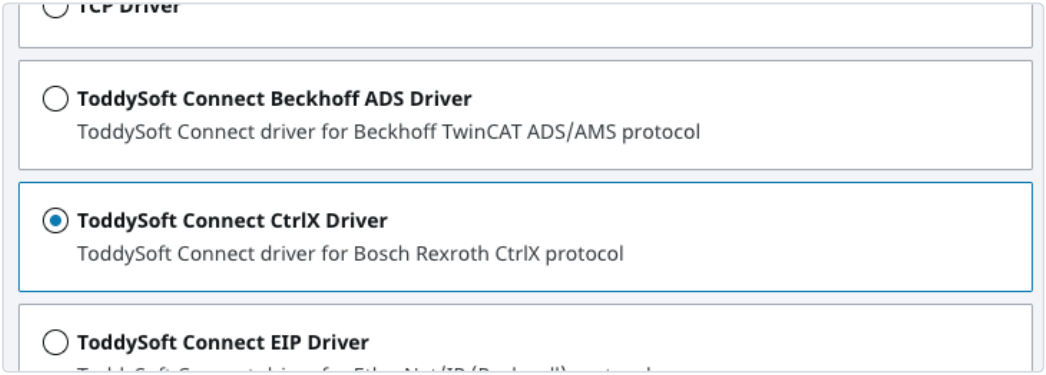
The certificate a ctrlX CORE ships with is self-signed and chains to no public CA, so ordinary verification cannot pass. See *Reaching a device with its factory certificate* below — the answer is to pin it, not to switch verification off and leave it off.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

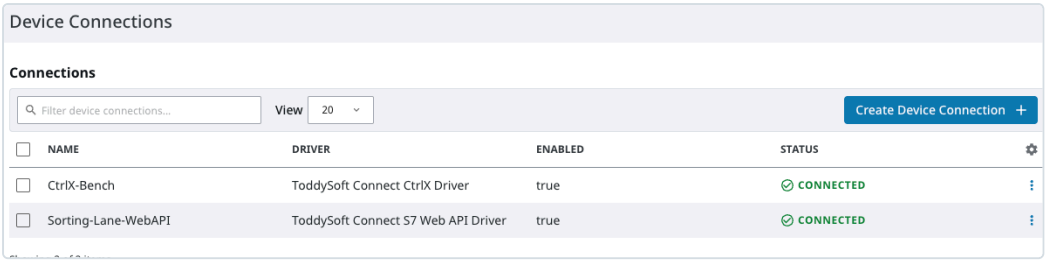


2. Click **Create new Device...** and select **ToddySoft Connect CtrlX Driver** as the device type.



3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

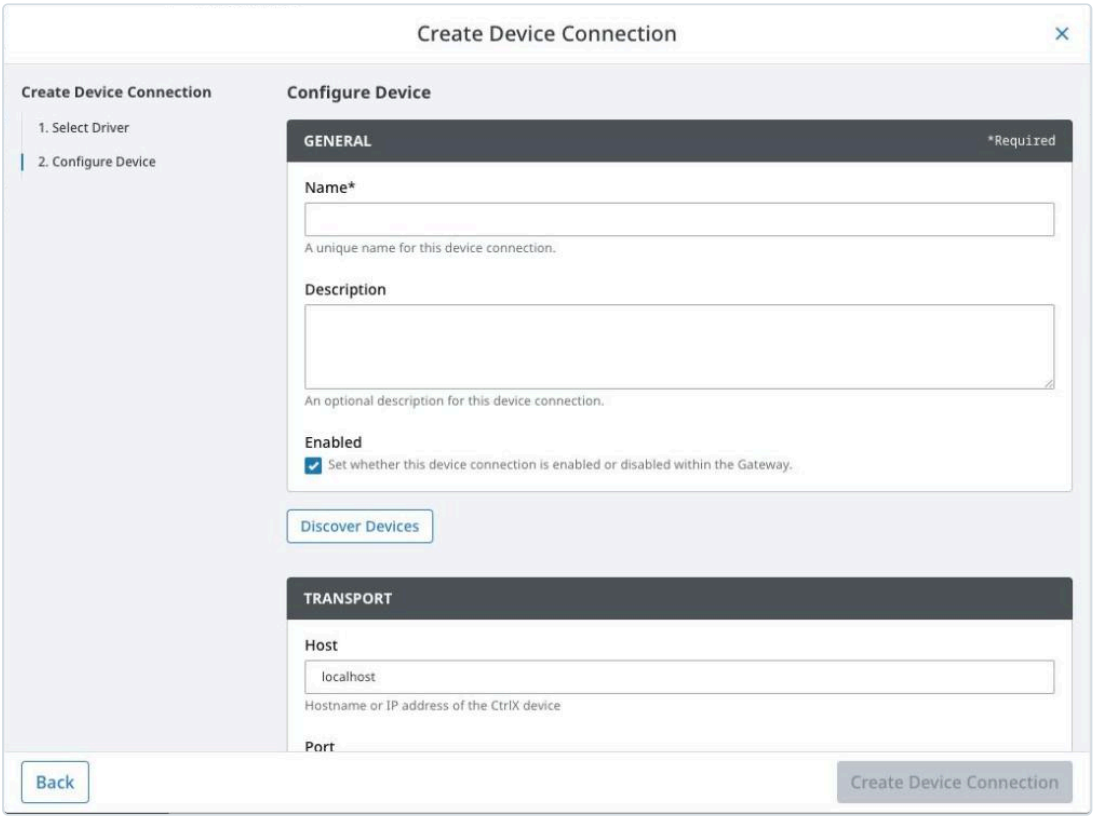
4. The new device then appears in the Device Connections list.



> **Ignition 8.3 can find ctrlX devices for you; Ignition 8.1 cannot.** The 8.1 device page has no discovery action for any driver — enter the host by hand there. Neither version offers a *Test Connection* action for ctrlX: create the device and read its status.

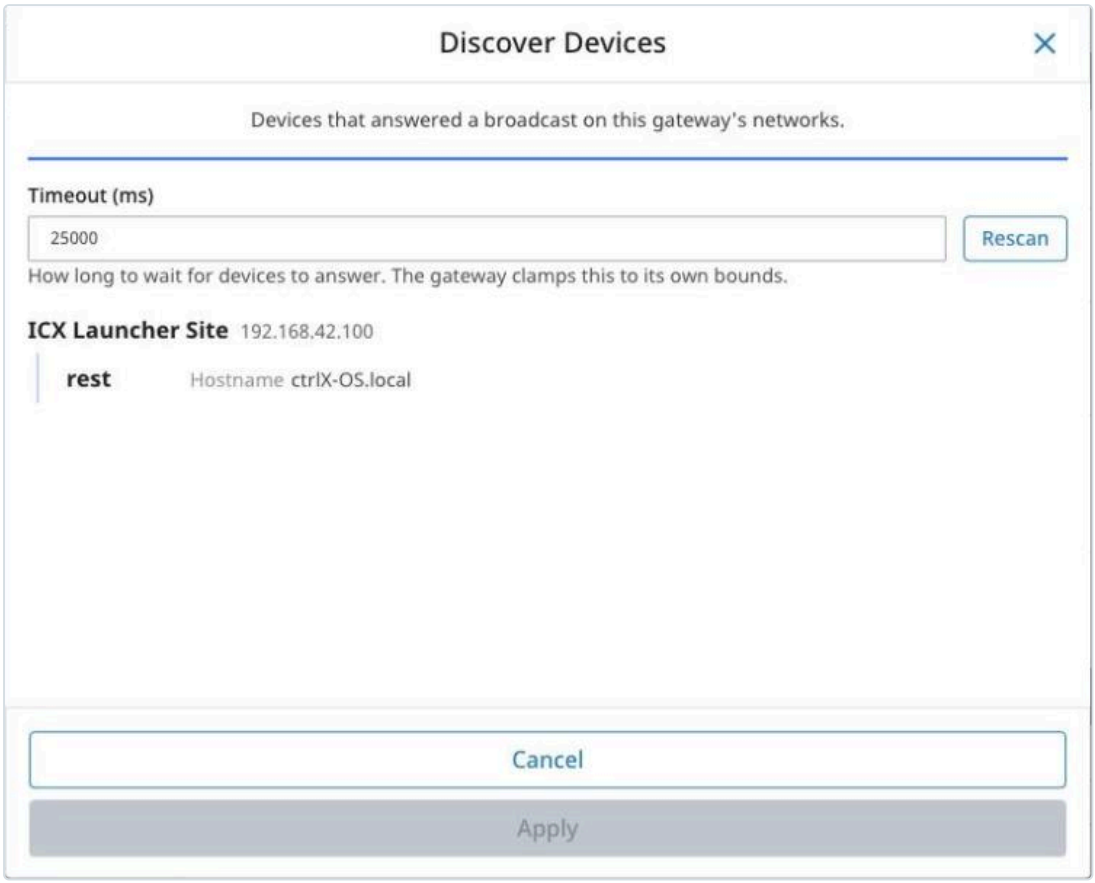
Finding devices

Above the configuration form sits **Discover Devices**. It searches the gateway's own networks and lists the ctrlX controllers that answer.

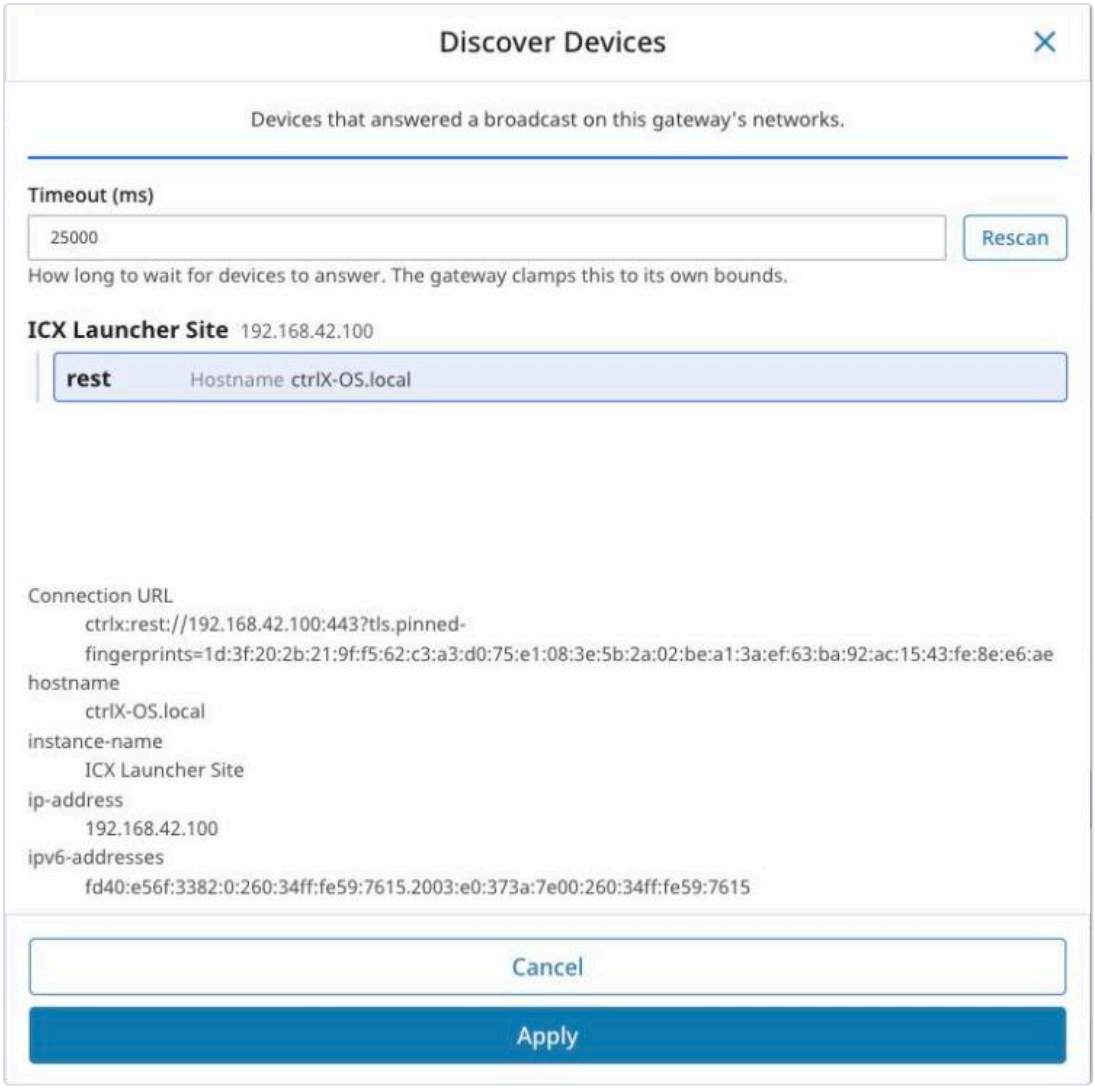


A ctrlX announces itself over mDNS, but nothing in that announcement identifies it *as* a ctrlX: the search asks the generic question every web-serving device on the network answers, and both the name and the hostname a ctrlX advertises are chosen by whoever commissioned it. So each responder is then asked one unauthenticated

question that only a ctrlX Data Layer answers the way a ctrlX does, and only the ones that confirm are listed. That is why the search takes a few seconds per network interface — and why a controller you renamed is still found.

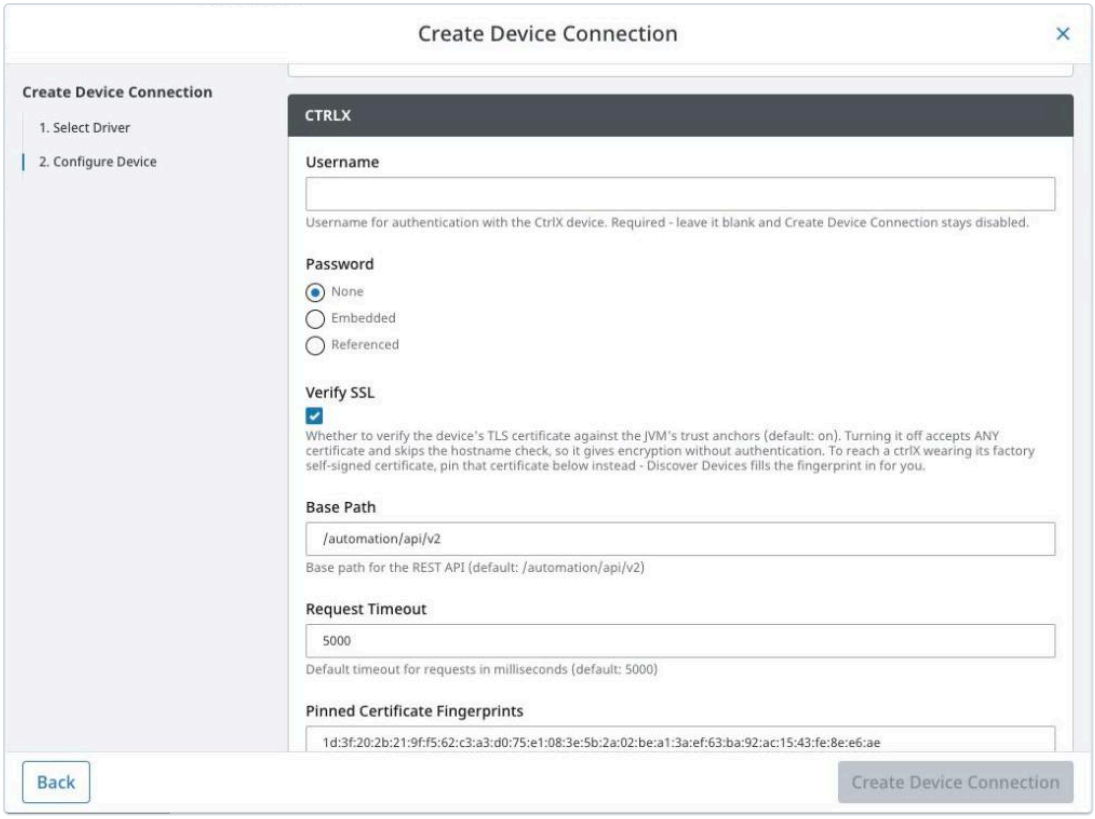


Selecting one shows what that device reported — the connection URL it would be reached on, its hostname and its addresses — before anything is written into the form.



Selecting a result fills in **Host**, **Port** and **Pinned Certificate Fingerprints**. Four things it deliberately does not do:

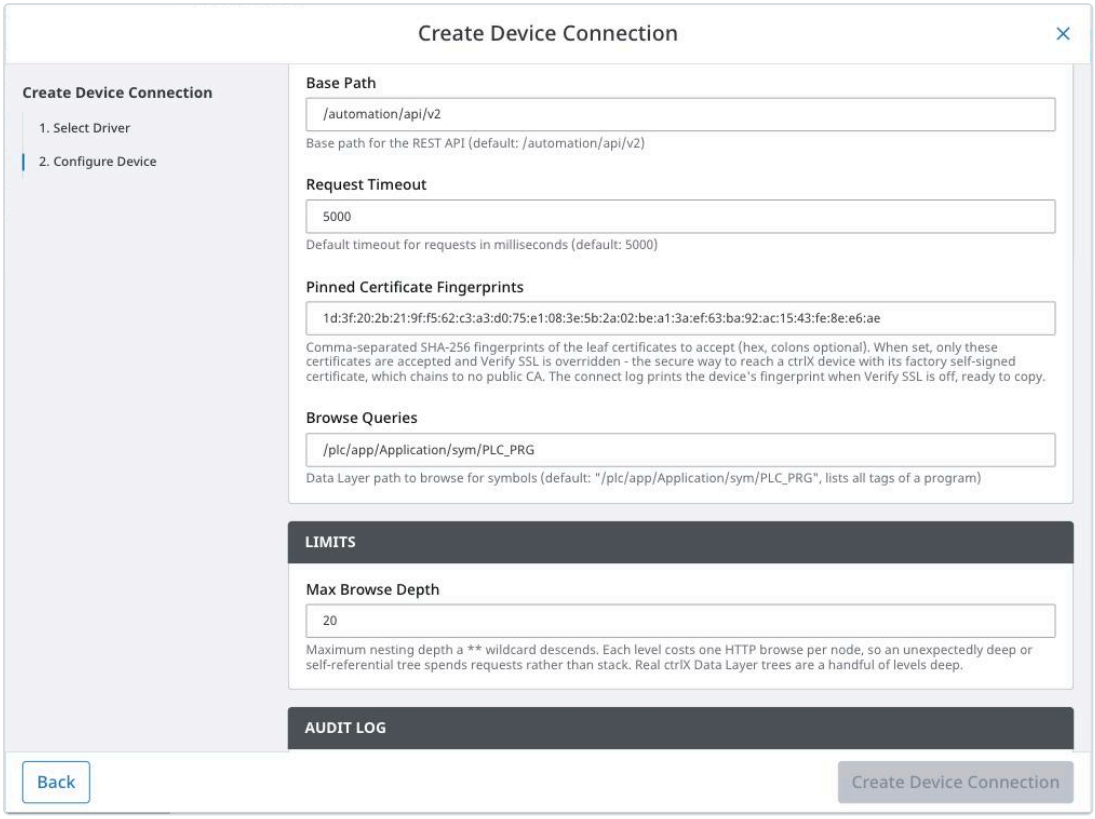
- **It never fills Username or Password.** A device that answers a search tells you where it is, never who may log in to it. ctrlX refuses every connection that arrives without credentials, so these two are always yours to enter — a device you found is not a device you can read until you have.
- **It does not fill in the port the device advertised.** A ctrlX advertises its ICX Launcher web site, typically 8089, which is not the Data Layer this driver talks to. The port filled in is the Data Layer's 443.
- **It does not turn Verify SSL off.** A search never weakens a security setting on your behalf; it pins the exact certificate it saw instead — see below.
- **It fills in only what was actually observed.** A field the search learned nothing about keeps the value you gave it, and if applying a result would overwrite something you have already typed, the form lists what would change and asks first.



Running a search needs permission to create device connections; being logged in is not enough.

The fingerprint a search fills in

Pinned Certificate Fingerprints is filled with the SHA-256 of every certificate the search actually saw on that device — a device may present several, one per connection at random, and the search connects repeatedly to collect them all. It is what makes a discovered ctrlX connectable at all: a factory certificate chains to no public CA, so without a pin the connection fails its TLS check against the very device that just answered.



Treat it as trust on first use. The fingerprint was read over a connection that could not yet be verified — which is the only way to read it the first time — so it is exactly as trustworthy as your network was at that moment. If you can compare it with the value the device's own web UI reports, do so. If you cannot, you still know precisely which certificate you are trusting, which is more than switching **Verify SSL** off would tell you.

Running in Docker: the discovery agent

A gateway in a Docker container sees only the container's networks, not the plant's: *Discover Devices* searches with mDNS, a multicast that never leaves the container's network, so no controller answers. The **ToddySoft discovery agent** closes that gap. It is a small, separate container that runs on the same machine with the host's network and searches the plant networks on the gateway's behalf. The agent only searches: connecting, browsing and reading stay with the gateway, through the machine's normal routing. A gateway installed directly on the plant machine, or a container on the host network, does not need it.

Installing it. The agent comes as an image archive next to this module, in the share you download the module from: `discovery-agent-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the agent of the module's version. On the machine that runs the Ignition container (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lets it search the plant networks, and `NET_RAW` lets it send and capture the discovery frames. Docker Desktop (macOS, Windows) runs containers in a virtual machine, whose networks the agent would search instead of the plant's.

Nothing to set in Ignition. On every **Discover Devices** the module looks for the agent at its container's default gateway, which on Docker is the machine itself. An agent started later is found on the next search, with no restart. A line above the results says what happened:

- *via discovery agent*: the agent searched; its devices are listed together with any the gateway found itself, each once.
- *no discovery agent found · container networks only*: no agent runs; the search is the gateway's own, as before.
- *discovery agent refused: …* or *discovery agent not usable: …*: the reason follows, for example the licence or the shared secret below.

Licence. The agent searches only for a gateway licensed for this driver. With each search the module sends the licence it runs on, the bundled trial included, over an encrypted connection; the agent checks it, uses it for that search and keeps nothing.

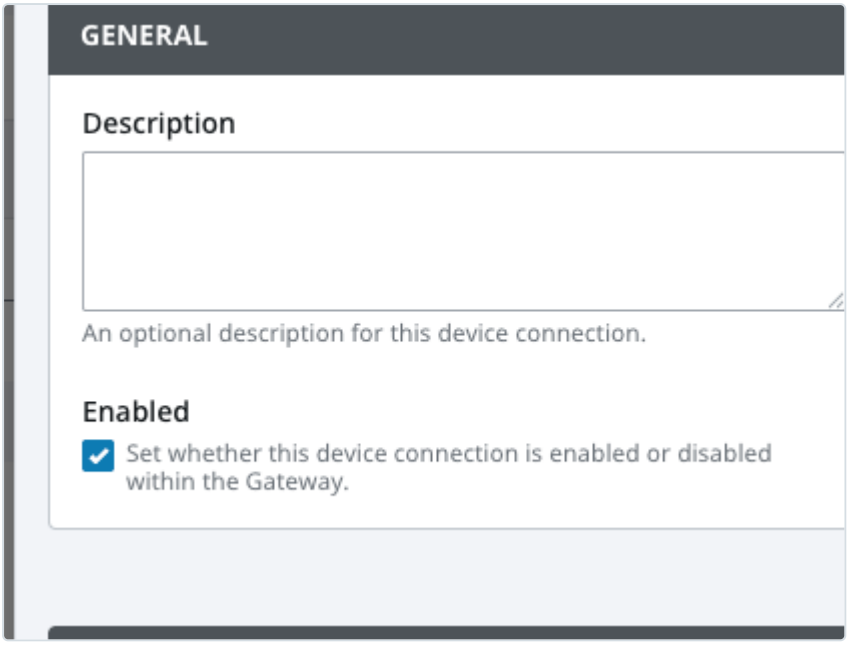
Security. The agent listens only on the Docker networks' own addresses on that machine (port 47900), never on a plant network, and accepts connections only from the container networks. To restrict it further, give it a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the Ignition container. With a secret set, a gateway without it, or with another, is refused. To stop the module looking for an agent, set `TODDYSOFT_DISCOVERY_AGENT=off` on the Ignition container.

Setting blocks

The configuration is organized into categories: **General**, **Transport**, **CtrlX**, **Limits** and **Audit Log**.

General

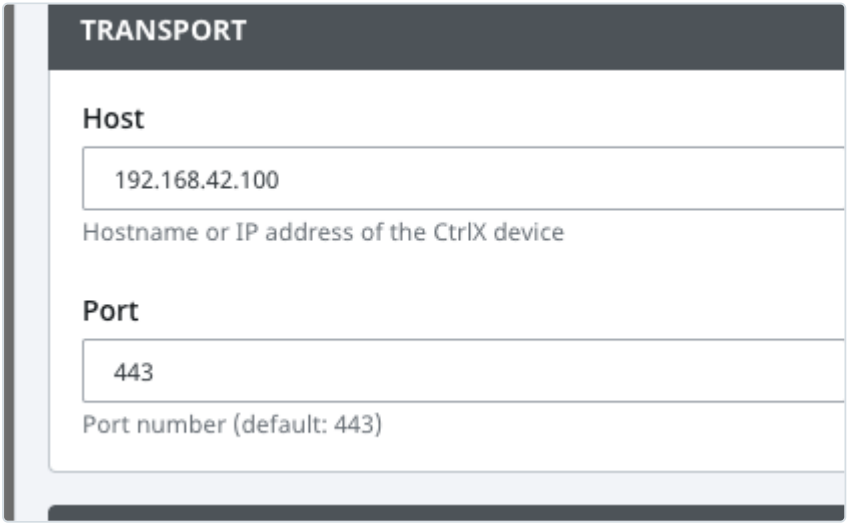
Assigns a name (and optional description) to the connection and controls whether it is enabled.



Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	`true`

This is the name the connection is then referenced under in other parts of Ignition.

Transport



Setting	Description	Default
Host	Hostname or IP address of the ctrlX device	
Port	HTTPS port of the Data Layer REST API	`443`

There is no transport selector, and none is missing: this driver speaks HTTPS and nothing else. The port is a plain override for a device behind a reverse proxy or a non-standard listener.

CtrlX

CTRLX

Username

boschrexroth

Username for authentication with the CtrlX device

Password

- ○ None
- ◉ Embedded
  - ☐ Update Password?

  Secret

  encrypted
- ○ Referenced

Verify SSL

☐

Whether to verify SSL certificates (default: false)

Base Path

/automation/api/v2

Base path for the REST API (default: /automation/api/v2)

Request Timeout

5000

Default timeout for requests in milliseconds (default: 5000)

Pinned Certificate Fingerprints

Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional). When set, only these certificates are accepted and Verify SSL is overridden - the secure way to reach a ctrlX device with its factory self-signed certificate, which chains to no public CA. The connect log prints the device's fingerprint when Verify SSL is off, ready to copy.

Browse Queries

/plc/app/Application/sym/PLC_PRG

Data Layer path to browse for symbols (default: "/plc/app/Application/sym/PLC_PRG", lists all tags of a program)

LIMITS

Setting	Description	Default
Username	Account the driver authenticates as	
Password	That account's password	
Verify SSL	Whether the device certificate is verified	`true`
Base Path	Base path of the REST API	`/automation/api/v2`
Request Timeout	Milliseconds allowed for one HTTP request	`5000`
Pinned Certificate Fingerprints	SHA-256 fingerprints of the exact leaf certificates to accept	
Browse Queries	Data Layer path(s) to browse for symbols; separate multiple queries with `;`	`/plc/app/Application/sym/PLC_PRG`

Username and password are both required. ctrlX authenticates every request and the driver refuses an unauthenticated connection. They are *not* part of the connection string: the driver takes them as a credential object, which is why they never appear in a logged URL.

The password is held in Ignition's secret store. The picker offers *None*, *Embedded* and *Referenced*: **Embedded** keeps the secret with this device, **Referenced** takes it from a configured secret provider, and *None* leaves the driver with no password — which this driver cannot connect with.

Reaching a device with its factory certificate

A ctrlX CORE's factory certificate is self-signed, so it chains to no public CA and names the device rather than its address. Ordinary verification cannot pass, and since Connect 1.2.0 `Verify SSL` also checks the hostname — which such a certificate fails on both counts.

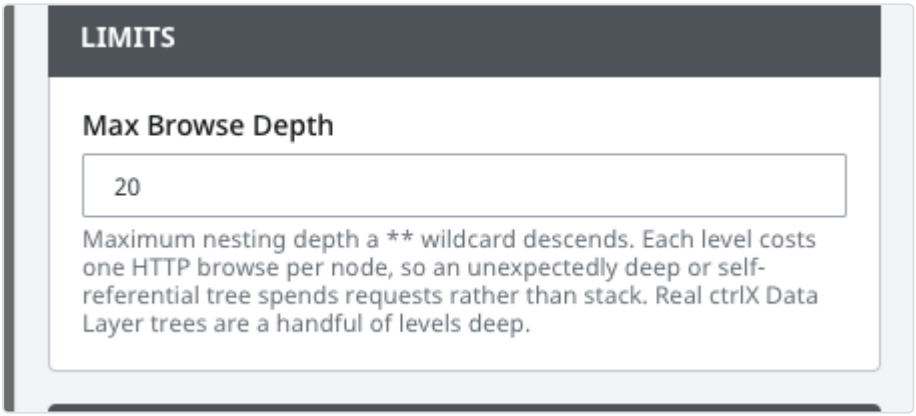
Pin it. Put the device's SHA-256 leaf fingerprint in **Pinned Certificate Fingerprints** (hex, colons optional; comma-separate several). A pin accepts exactly the certificate you listed and refuses a substituted one even though that one is equally self-signed, which is strictly stronger than turning verification off. Pinning **overrides Verify SSL**, and the hostname is deliberately not checked in that mode — the pin already identifies the peer.

The easiest way to obtain the fingerprint is not to turn anything off. Press *Discover Devices* and pick the device: the search reads the certificate itself and fills the field in, leaving **Verify SSL** alone — see *Finding devices*. Failing that, connect once with **Verify SSL** off and the driver logs the device's SHA-256 ready to copy, then paste it into the field. Where your site runs its own CA, provisioning a real certificate remains the better answer.

A device may present more than one certificate. Some ctrlX controllers hold several self-signed certificates and answer each connection with one of them at random; the ctrlX CORE on our bench has two. The pin list must name every one, or the connections that draw a missing one are refused — intermittently, which looks like a network fault. When that happens the driver's log says `TLS pin check failed: peer certificate SHA-256 fingerprint <value> is not in the pinned list`: **add** that value to the field, comma-separated, rather than replacing what is there. A search avoids this: it fills in every certificate it saw.

`Verify SSL` off accepts any peer at all, which is why it is a starting point for getting the fingerprint and not a resting place.

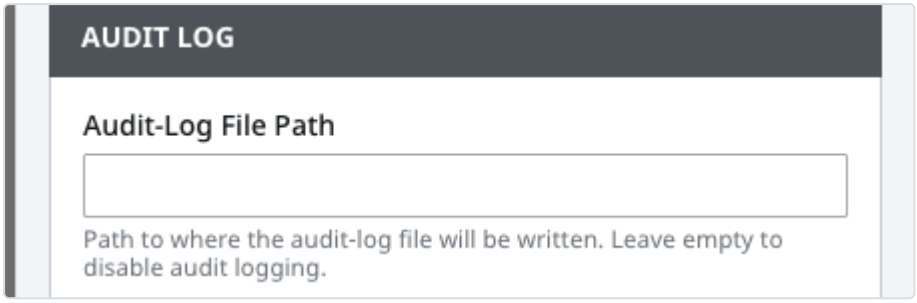
Limits



Setting	Description	Default
Max Browse Depth	How far a `**` wildcard descends	`20`

Each level a `**` wildcard descends costs one HTTP browse per node, so an unexpectedly deep — or self-referential — subtree spends requests rather than stack. Real Data Layer trees are a handful of levels deep, and the default of `20` is far above anything legitimate; it is a backstop, not a tuning knob.

Audit Log



Setting	Description	Default
Audit-Log File Path	Path to a file protocol audit events are written to; empty disables	

The audit log records protocol-level events and grows quickly, so enable it only while investigating a problem.

Tag Addressing

ctrlX addresses are Data Layer node paths — slash-separated, like a filesystem, with or without a leading slash:

```
<segment>(/<segment>)*
```

Address	Meaning
`plc/app/Application/sym/PLC_PRG/myVariable`	A PLC variable exposed through the Data Layer
`motion/axs/MyAxis/state/values/actual/pos`	An axis's actual position
`framework/metrics/system/cpu-utilisation-percent`	A single CPU-utilisation node
`framework/metrics/system/**`	Pattern — every descendant of that namespace (browsing only)

A leading `/` is optional: `plc/app/…` (Bosch's spelling) and `/plc/app/…` (what browsing shows) name the same node. Whitespace, a `?query` or an empty segment (`//`) gives `INVALID_ADDRESS` for that tag alone; the other tags of its group are still read.

The Data Layer tree is large. Restrict **Browse Queries** to the subtree you actually want — the default, `/plc/app/Application/sym/PLC_PRG`, lists the symbols of one PLC program — rather than browsing from the root.

Types

Data Layer nodes carry their type in the response payload (`int8`, `uint16`, `float64`, `string`, `bool`, …), and the driver maps that to the matching OPC UA type. Nothing has to be declared in the address.

A node the controller reports without a usable type is skipped, with a `WARN` naming it, and the rest of the browse is unaffected — one untypeable node does not cost the device.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

This matters more here than on a fieldbus driver. Each tag group costs one HTTPS request per poll, however many tags it holds, so ten groups at one second are ten requests a second against a management interface. Put slow-moving diagnostics in a slower group rather than in many fast ones.

Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open the device's settings (**Config > OPC UA > Device Connections**, edit the device, or create a new one): the **Licence** card at the top of the form shows the diagnosis. It is on the form for a new device too, so the licence can be checked before the first device exists.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

Cause	Meaning	What to do
`IGNITION_UNLICENSED`	Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence.	Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation. If a key was activated and this cause remains, see Troubleshooting: an older, expired key covering this module may be the one Ignition judges it by.
`NO_DATA_FOR_DRIVER`	The activated key carries no ToddySoft licence for this driver.	Send the report to ToddySoft.
`DATA_INCOMPLETE`	The licence data arrived incomplete or damaged.	Send the report to ToddySoft; the licence has to be delivered again.
`DATA_REJECTED`	The driver refused the delivered licence; the reason is shown.	Send the report to ToddySoft.
`RUNNING_ON_BUNDLED_TRIAL`	A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends.	Send the report to ToddySoft.
`NOT_CHECKED`	Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded.	Send the report to ToddySoft.
`VALID`	Licensed; tier and expiry are shown.	Nothing.

Cause	Meaning	What to do
`NOT_YET_KNOWN`	The first seconds after start-up, before Ignition reports a licence state.	Wait a moment.

Sending a report to support. The diagnosis comes as a plain-text report, the one thing to send with any licence question. **Copy report** puts it on the clipboard; **Show report** displays it. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.ctrlx/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

Troubleshooting

Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index", or `IGNITION_UNLICENSED` with "trial expired", in the licence diagnosis:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one. Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.
- If one of the two keys has **expired**, Ignition may judge the module by that key alone: the diagnosis shows `IGNITION_UNLICENSED` ("trial expired - no activated licence applies to this module") and "Data: none received", although the new key was activated successfully.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Platform > Licensing**, then restart the module (or the gateway). In a redundant pair, check every node.

The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

Device shows "Faulted" or sits in "Reconnecting":

- Verify the device is reachable over HTTPS on the configured port — `curl -k https://<host>/` is enough to tell a network problem from a certificate one
- Review Gateway logs: **Status > Diagnostics > Logs**

`SSL handshake failed` although the device answers:

- The factory certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Certificate Fingerprints** — see *Reaching a device with its factory certificate*

HTTP 401 Unauthorized:

- Wrong username or password, or the account was left empty

HTTP 403 Forbidden — reads work, writes are refused:

- The account authenticated, but lacks the role the operation needs. ctrlX manages read and write as separate permissions; adjust the role in the controller's user management

HTTP 404 Not Found:

- The node path does not exist. Browse to discover legal paths rather than typing them

`INVALID_ADDRESS` from a read or write:

- Whitespace, a `?query` or an empty segment in the address; a leading `/` is fine. Only that tag fails, the rest of its group is read

Reads are slow or time out:

- Each tag group is one HTTPS request per poll, answered by the controller's web server. Give slow-moving tags a slower group, and raise **Request Timeout** for a controller under load

Docker related issues

The ctrlX protocol carries no identity for this end of the connection, so a container's internal address is never announced to the device and nothing has to be reconciled. A gateway in a bridged container connects normally, provided the device's HTTPS port is routable.

Device discovery needs the discovery agent. *Discover Devices* searches the networks the gateway itself is attached to, which for a container are Docker's, not the plant's. Run the ToddySoft discovery agent next to the container — see Running in Docker: the discovery agent — or enter the host by hand.

VPN related issues

A remote device reached over a VPN needs nothing special from this driver, for the same reason: the connection is an ordinary outbound HTTPS session. Only reachability and the certificate matter.

Version history

Unreleased — 1.3.0

- **Changed:** a tag address is accepted with and without a leading `/` (`plc/app/…`, Bosch's spelling, and `/plc/app/…`, the browser's); until now the form without it was refused. An address the driver refuses now fails that tag alone, not the whole read of its group.
- **Changed: Verify SSL** now defaults to **on**, matching the driver's own default. It had shipped off, so a device nobody had adjusted accepted any certificate at all and skipped the hostname check — encryption without authentication. A ctrlX wearing its factory certificate is reached by pinning that certificate, not by leaving verification off for every device; if you rely on the old behaviour, the checkbox is still there.
- **Added: Discover Devices** on Ignition 8.3 — the device page can now search the gateway's networks for ctrlX controllers and list the ones that confirm themselves, so the address no longer has to be typed from a commissioning note. Picking a result fills in the host, the Data Layer port and the pinned fingerprints of every certificate that device presented; it never fills a username or password, and never turns **Verify SSL** off. Requires permission to create device connections, and does not work from inside a bridged container.
- First release of this manual.
- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.
- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.
- **New:** discovery when Ignition runs in Docker, through the ToddySoft discovery agent. See Running in Docker: the discovery agent.
- **Changed:** date and time tags have new value types. DATE and DATE_AND_TIME are DateTime values (until now a DateTime tag holding epoch milliseconds as a number); TIME and LTIME are milliseconds, TIME_OF_DAY and LTIME_OF_DAY milliseconds since midnight (Int64). Writes take the same forms. Adjust scripts and bindings that used the old values.
- **Fixed:** a write the PLC refuses now fails in Ignition, with the reason as the status (for example `Bad_TypeMismatch` for a value of the wrong type). Until now every write was reported as successful, and the refusal appeared only in the gateway log.

- **Fixed:** a tag that has not been read yet, or whose last read failed, no longer reads 0 with Good quality. Before its first read it has no value and the quality Uncertain; after a failed read it keeps its last value with Bad quality until the next good read.
- **Fixed:** an LTIME tag reads in milliseconds, like TIME; the ctrlX Data Layer reports it in nanoseconds, so 3.2 s read as 3200050000. Writes to LTIME and to unsigned integer tags (UDINT, UINT, USINT, which include TIME and TIME_OF_DAY) now reach the controller; the controller refused them.
- **Fixed:** removing the last tag of a tag group (or polling rate) no longer leaves the device showing *Connection Lost*. A poll still running at that moment made the device fail with `Batch '...' not found` in the gateway log and stay in that state until it was restarted.
- **Changed:** a browsed array is one folder, named after the array and its index range (for example `arr[0..9]`), holding its elements; until now the elements sat directly beside the array's neighbours. A multi-dimensional array, until now left out of the browse, appears as nested folders down to its elements (`cube[1..2][1..2]`, then `cube[1][1..2]`, then `cube[1][1]`). An array of structures shows a folder per element holding its members (for example `pts[2].x`). Each element keeps its OPC item path, so existing tags keep working.